Travaux dirigés : construire l'API CRUD des documents

Objectif

Vous allez créer cinq routes et un contrôleur pour lire, ajouter, modifier et supprimer les informations des documents dans PostgreSQL. Le résultat sera une API qui échange des données JSON.

Opération	Méthode	Adresse	Méthode du contrôleur
Lister	GET	`/api/documents`	`index`
Consulter	GET	`/api/documents/{document}`	`show`
Ajouter	POST	`/api/documents`	`store`
Modifier	PUT	`/api/documents/{document}`	`update`
Supprimer	DELETE	`/api/documents/{document}`	`destroy`

`{document}` représente l'identifiant du document. Pour ce TD, une modification par `PUT` demande le titre **et** la catégorie.

Avant de commencer

- Le projet et Docker ont été installés, puis le TD sur les migrations et le modèle a été réalisé. La table `documents` doit exister dans PostgreSQL.
- Ouvrez dans VS Code le dossier du projet, celui qui contient `artisan` et `compose.yaml`, puis ouvrez **Terminal > Nouveau terminal** et choisissez PowerShell. Les commandes ci-dessous s'y exécutent ; `curl.exe` est l'outil de test disponible sous Windows.
- Partez de la branche **`jour2-postgresql-migrations-modeles`** : elle contient les fichiers nécessaires au TD. La branche `jour2-api-crud-documents` contient la correction.

Vérifiez votre branche :

```
git branch --show-current
```

Si vous devez rejoindre la branche de départ :

```
git switch jour2-postgresql-migrations-modeles
```

Si Git signale des modifications ou des fichiers non suivis qui empêchent le changement, conservez votre travail et demandez au formateur comment le sauvegarder. Ne forcez pas le changement et ne supprimez pas vos fichiers. Si la branche est introuvable dans votre copie du dépôt GitHub, exécutez `git fetch origin`, puis réessayez `git switch jour2-postgresql-migrations-modeles`.

Vérifiez les conteneurs et la table :

```
docker compose ps
docker compose exec app php artisan migrate:status
docker compose exec app php artisan db:table documents
```

À vérifier : les services `app` et `db` sont démarrés, la migration `create_documents_table` est indiquée `Ran`, et la table contient notamment `id`, `title` et `category`. Si la table n'existe pas encore, terminez le TD précédent avant de poursuivre. Les migrations s'exécutent pendant le cours, pas automatiquement au démarrage de Docker.

1. Créer le contrôleur

Dans le terminal du projet, lancez :

```
docker compose exec app php artisan make:controller DocumentController --api
```

Laravel crée `app/Http/Controllers/DocumentController.php`. L'option `--api` prépare un contrôleur adapté aux opérations d'une API. Si le fichier existe déjà, ne le générez pas une deuxième fois : vérifiez votre branche et demandez l'aide du formateur.

2. Écrire le contrôleur

Dans l'explorateur VS Code, ouvrez **`app/Http/Controllers/DocumentController.php`**. Remplacez tout son contenu par le code suivant et enregistrez avec **Ctrl+S** :

```
<?php

namespace App\Http\Controllers;

use App\Models\Document;
use Illuminate\Http\JsonResponse;
use Illuminate\Http\Request;
use Illuminate\Http\Response;

class DocumentController extends Controller
{
    public function index(): JsonResponse
    {
        return response()->json(Document::all());
    }

    public function show(Document $document): JsonResponse
    {
        return response()->json($document);
    }

    public function store(Request $request): JsonResponse
    {
        $data = $request->validate([
            'title' => ['required', 'string', 'max:100'],
            'category' => ['required', 'string', 'max:50'],
        ]);

        $document = Document::create($data);

        return response()->json($document, 201);
    }

    public function update(Request $request, Document $document): JsonResponse
```

```
    {
        $data = $request->validate([
            'title' => ['required', 'string', 'max:100'],
            'category' => ['required', 'string', 'max:50'],
        ]);

        $document->update($data);

        return response()->json($document);
    }

    public function destroy(Document $document): Response
    {
        $document->delete();

        return response()->noContent();
    }
}
```

Les méthodes `index` et `show` lisent les données à l'aide du modèle `Document`. Dans `show`, `update` et `destroy`, Laravel recherche automatiquement le document dont l'identifiant est fourni dans l'adresse : si aucun document ne correspond, il renvoie une réponse **404**.

`store` et `update` vérifient que `title` et `category` sont présents, sont des chaînes et respectent les limites de la table : respectivement 100 et 50 caractères. Seules les données validées sont utilisées. `store` répond **201** après une création ; `update` répond **200** après une modification. `destroy` supprime la ligne puis répond **204**, sans contenu.

Comprendre la validation

Le navigateur peut vérifier un formulaire avant l'envoi pour aider l'utilisateur. Mais une requête peut aussi être envoyée directement avec `curl.exe` ou un autre programme : elle ne passe alors pas par le formulaire. **Laravel doit donc vérifier chaque requête reçue**, même si le navigateur effectue déjà des vérifications.

Dans `store` et `update`, `$request->validate(...)` applique les mêmes règles :

Champ	Règles	Signification
`title`	`required`, `string`, `max:100`	Obligatoire, texte de 100 caractères au maximum.
`category`	`required`, `string`, `max:50`	Obligatoire, texte de 50 caractères au maximum.

Si les données sont valides, Laravel renvoie les valeurs validées dans `$data`, utilisées ensuite par `create` ou `update`. Si elles sont invalides, la méthode de validation interrompt le traitement : la création ou la modification n'est pas exécutée. Pour une requête qui attend du JSON, Laravel répond **422** avec les erreurs associées aux champs.

```
Navigateur ou curl → requête JSON → Laravel vérifie les règles
                                    ├─ valide → création ou modification
                                    └─ invalide → réponse 422 et erreurs JSON
```

Les règles sont ici écrites directement dans le contrôleur, une fois pour la création et une fois pour la modification. Le navigateur pourra appliquer des vérifications supplémentaires pour faciliter la saisie ; cela ne modifie pas les règles de l'API.

Lire les réponses HTTP

Statut	Signification dans ce TD
200 OK	Lecture ou modification réussie.
201 Created	Document créé.
204 No Content	Suppression réussie, sans corps de réponse.
404 Not Found	Aucun document ne correspond à l'identifiant demandé.
422 Unprocessable Content	Les données envoyées ne respectent pas les règles de validation.

Les étapes suivantes permettent d'observer ces réponses. L'en-tête `Accept: application/json` demande une réponse JSON, notamment pour les erreurs de validation ; `Content-Type: application/json` décrit le format des données envoyées.

3. Déclarer les routes

Ouvrez **`routes/api.php`** dans VS Code. Remplacez tout le contenu par le code suivant, puis enregistrez :

```
<?php

use App\Http\Controllers\DocumentController;
use App\Http\Controllers\DocumentExempleController;
use Illuminate\Support\Facades\Route;

Route::get('/exemples/document', [DocumentExempleController::class, 'show']);

Route::get('/documents', [DocumentController::class, 'index']);
Route::get('/documents/{document}', [DocumentController::class, 'show']);
Route::post('/documents', [DocumentController::class, 'store']);
Route::put('/documents/{document}', [DocumentController::class, 'update']);
Route::delete('/documents/{document}', [DocumentController::class, 'destroy']);
```

La première route, déjà présente, reste un exemple fictif. Les cinq nouvelles routes appellent les méthodes de `DocumentController`. Laravel ajoute automatiquement le préfixe `/api` aux chemins déclarés dans ce fichier.

Vérifiez les routes enregistrées :

```
docker compose exec app php artisan route:list --path=api
```

À vérifier : vous voyez les cinq routes `documents` et la route d'exemple. Pour le document unique, le nom `{document}` correspond au paramètre `Document $document` du contrôleur.

4. Préparer les données des requêtes

Dans VS Code, créez un fichier **`document.json`** à la racine du projet, au même niveau que `compose.yaml`. Ajoutez ce contenu et enregistrez :

```
{
  "title": "Contrat de formation",
```

```
  "category": "Administratif"
}
```

Ce fichier sert uniquement à fournir un corps JSON aux commandes de test. Le titre et la catégorie représentent des informations sur un document ; aucun fichier n'est téléversé dans l'application.

5. Ajouter un document

Exécutez cette commande PowerShell dans le dossier du projet :

```
curl.exe -i -X POST -H "Accept: application/json" -H "Content-Type: application/json" --data-binary "@document.json"
http://localhost:8088/api/documents
```

`Content-Type` indique que le corps est en JSON. `Accept` demande une réponse JSON, y compris pour les erreurs de validation. `-i` affiche le statut HTTP et les en-têtes.

À vérifier : la réponse indique **201 Created** et contient l'objet créé avec un `id`. Notez cet identifiant pour les étapes suivantes. Si vous relancez la commande, vous créez une nouvelle ligne.

Dans pgAdmin, ouvrez la table **documents**, puis faites un clic droit sur celle-ci et choisissez **View/Edit Data > All Rows**. La ligne créée apparaît avec son `id`, son titre et sa catégorie. Pour configurer l'accès à pgAdmin, suivez l'étape 8 du guide de démarrage.

6. Lister et consulter

Pour afficher la liste :

```
curl.exe -i -H "Accept: application/json" http://localhost:8088/api/documents
```

À vérifier : la réponse indique **200 OK** et contient un tableau JSON avec le document créé. Une liste vide est représentée par `[]`.

Pour consulter un document, adaptez l'identifiant dans l'adresse. Dans cet exemple, `1` est un identifiant d'exemple : remplacez-le par celui obtenu à l'étape 5 s'il est différent.

```
curl.exe -i -H "Accept: application/json" http://localhost:8088/api/documents/1
```

À vérifier : la réponse indique **200 OK** et contient un objet JSON avec le titre et la catégorie du document choisi.

7. Modifier un document

Dans `document.json`, remplacez le contenu par les deux champs suivants, puis enregistrez :

```
{
  "title": "Contrat mis a jour",
  "category": "Formation"
}
```

Adaptez l'identifiant `1` si nécessaire, puis lancez :

```
curl.exe -i -X PUT -H "Accept: application/json" -H "Content-Type: application/json" --data-binary "@document.json"
http://localhost:8088/api/documents/1
```

À vérifier : la réponse indique **200 OK** et présente les nouvelles valeurs. Relancez le `GET` de l'étape 6 : le changement est toujours présent dans la base.

8. Tester les erreurs

Testez un identifiant qui n'existe pas dans votre base. L'exemple suivant utilise `999999` : vérifiez dans la liste qu'il est absent, ou choisissez un autre identifiant absent.

```
curl.exe -i -H "Accept: application/json" http://localhost:8088/api/documents/999999
```

À vérifier : la réponse indique **404 Not Found**.

Pour tester la validation, créez à la racine un fichier `document-invalide.json` contenant un objet vide :

```
{}
```

Lancez :

```
curl.exe -i -X POST -H "Accept: application/json" -H "Content-Type: application/json" --data-binary "@document-
invalide.json" http://localhost:8088/api/documents
```

À vérifier : la réponse indique **422 Unprocessable Content** et son JSON contient des erreurs pour `title` et `category`. Les textes peuvent être en anglais selon la configuration. Essayez aussi un `PUT` avec un champ manquant : le document existant ne doit pas être modifié.

9. Supprimer le document de test

Utilisez l'identifiant créé à l'étape 5, en remplaçant `1` si nécessaire :

```
curl.exe -i -X DELETE -H "Accept: application/json" http://localhost:8088/api/documents/1
```

À vérifier : la réponse indique **204 No Content** et ne contient pas de corps. Un nouveau `GET` sur cet identifiant renvoie **404**. La suppression efface la ligne de la base.

10. Questions de compréhension

1. Quel fichier associe une méthode HTTP et une adresse à une méthode du contrôleur ?
2. Quel est le rôle du modèle `Document` dans le contrôleur ?
3. Pourquoi faut-il valider `title` et `category` avant l'enregistrement ?
4. Quelle différence y a-t-il entre les réponses 201, 204, 404 et 422 ?
5. Pourquoi `PUT` demande-t-il ici les deux champs, même si un seul change ?

Comparez ensuite votre travail avec la branche de correction `jour2-api-crud-documents`, selon les indications du formateur. Sauvegardez d'abord vos changements : Git peut refuser le changement de branche si les fichiers se chevauchent.

En cas de difficulté

Situation	Vérification
`no configuration file provided`	Le terminal est-il dans le dossier contenant `compose.yaml` ?
Connexion refusée sur le port 8088	Vérifiez Docker Desktop, `docker compose ps` et les journaux du service `app`.
`404` sur toutes les routes `documents`	Vérifiez l'adresse `/api/documents`, l'enregistrement du fichier `routes/api.php` et `route:list --path=api`.
Table `documents` absente	Terminez le TD précédent et vérifiez la connexion, la migration et la table.
`422` inattendu	Vérifiez le JSON, les deux noms de champs, leurs valeurs et leurs longueurs.
`curl` ne trouve pas le fichier JSON	Vérifiez son nom et lancez la commande depuis le dossier où il se trouve, à la racine du projet.
Réponse d'erreur HTML	Ajoutez l'en-tête `Accept: application/json` à la requête.